

Advanced Data Management

Week 5: Document Databases, Flexible Schemas & APIs
Bachelor of Computer Science · 4-Hour Lecture

Agenda — What We Cover Today

- MODULE 1 — Document Databases: JSON, BSON, and storage philosophy
- MODULE 2 — Schema Flexibility: Versioning and evolutionary design
- MODULE 3 — Indexing & Querying: Performance at scale
- MODULE 4 — Aggregation & Analytics: Computing results from data
- MODULE 5 — APIs: REST, CRUD, requests, and authentication

From Tables to Documents

- ▶ Traditional: data split across many tables
- ▶ Multiple JOIN operations reconstruct one object
- ▶ Schema is defined before data exists
- ▶ Optimised for storage efficiency
- ▶ Query complexity grows with relationships

- ▶ Modern: one document per entity
- ▶ All related data stored together — no joins
- ▶ Schema reflects real application objects
- ▶ Optimised for read performance
- ▶ Single collection query returns complete data

The Impedance Mismatch Problem

- Code works with objects: Student has a list of Courses
- Relational DB works with rows: a student_courses join table
- Every read requires translating rows back to objects
- ORMs (Hibernate, SQLAlchemy) automate translation but add overhead
- Document DBs store and return objects as-is — no translation needed



MODULE 1

Document Databases — JSON, BSON & Storage Philosophy

What is a Document Database?

- Stores data as self-contained documents
- One document describes one entity completely
- All relevant data kept together — no splits, no joins
- Documents use JSON format (or binary BSON internally)
- Philosophy: store data as the application sees it

Document Thinking

- Ask first: how will the application use this data?
- Design around access patterns, not normalisation rules
- If the UI needs profile + courses together, store them together
- The database adapts to the application's shape — not vice versa
- Duplicate data is acceptable if it saves expensive read operations

One Document = One Object

- A document corresponds directly to a programming object
- Student in code → Student document in the database
- Complete data returned in a single database read
- No JOIN statements required in application code
- Document databases complement, not replace, relational systems

What is JSON?

- JSON: JavaScript Object Notation
- A lightweight text format for representing structured data
- Human-readable — developers can understand it at a glance
- Language-independent — every programming language parses it
- The universal language of web APIs and document databases

Structure of JSON

- Objects defined with curly braces { } — contain key:value pairs
- Arrays defined with square brackets [] — contain ordered lists
- Keys must be strings in double quotes
- Values can be strings, numbers, booleans, arrays, or objects
- Any level of nesting is allowed: objects inside arrays inside objects

Basic JSON Data Types

- string: text in double quotes — "Maria Papadopoulou"
- number: integer or decimal — 8.5 or 12345
- boolean: true or false — no quotes, always lowercase
- null: explicitly absent value — intentionally unknown
- array and object: composite types that contain other values

Nested Structures in JSON

- Objects can contain other objects as field values
- Arrays can contain objects — a list of nested entities
- Unlimited depth: model any real-world hierarchy
- Replaces foreign keys and JOIN tables with embedded data
- Performance benefit: one disk read returns the complete hierarchy

Simple vs Nested Document

- ▶ Simple Document

- ▶ student_id: '12345'

- ▶ name: 'Maria'

- ▶ email: 'maria@example.com'

- ▶ Looks like a standard database row

- ▶ Nested Document

- ▶ student_id: '12345'

- ▶ name: 'Maria'

- ▶ courses: [{ name: 'NoSQL', grade: 8.5 }]

- ▶ Courses embedded — no separate table needed

JSON vs BSON

▶ JSON

- ▶ Text format — human readable
- ▶ Perfect for API transmission
- ▶ No native Date or Binary type
- ▶ Larger file size on disk

▶ BSON (Binary JSON)

- ▶ Binary format — machine readable
- ▶ Used internally by databases (MongoDB)
- ▶ Adds Date, Binary, and ObjectId types
- ▶ Faster parsing and smaller on disk

Popular Document Databases

- MongoDB: most popular, mature ecosystem, Atlas cloud service
- CouchDB: built-in replication, HTTP API, offline-first design
- RavenDB: developer-friendly, auto-indexing, .NET-first
- Firebase Firestore: Google-managed, real-time sync for mobile apps
- Amazon DocumentDB: managed MongoDB-compatible service on AWS

CouchDB — What It Is

- Built on Erlang — the language designed for telecom systems that never go down
- Every database operation is a plain HTTP request — no special driver needed
- Documents are pure JSON; the database speaks the web's native language
- Open-source Apache project since 2005; used by BBC, npm registry, NHS
- Philosophy: simplicity, reliability, and distributed-first design

CouchDB — Offline First & Replication

- Replication built in: sync any CouchDB instance to any other automatically
- Bidirectional sync: changes flow both ways — server to device and back
- Conflict detection: if the same document changes in two places, both versions are kept
- MVCC: readers and writers never block each other — no locking
- Use case: apps that must work without internet and sync when reconnected

RavenDB — What It Is

- Created in 2010 by Oren Eini — built out of frustration with complex NoSQL configuration
- The only major document DB with first-class .NET and C# support from day one
- Auto-indexing: RavenDB watches your queries and creates indexes automatically
- Goal: developer is productive on day one, no deep database expertise required
- Used in enterprises needing NoSQL flexibility with relational-grade reliability

RavenDB — Key Features

- ACID across multiple documents — guarantees consistency for complex operations
- Full-text search built in via Lucene — no separate search service required
- Document compression: stores data compressed on disk, reducing storage costs
- Subscriptions: push live document changes to connected services automatically
- Strong C# and Java SDKs with session-based unit-of-work pattern

Choosing Your Document Database

- ▶ MongoDB — best general-purpose choice
- ▶ Largest ecosystem, most tutorials, Atlas cloud
- ▶ Use when: any web/mobile app; no special constraints
- ▶ CouchDB — best for offline & sync
- ▶ Use when: app must work without internet; bidirectional sync required

- ▶ RavenDB — best for .NET enterprise teams
- ▶ Auto-indexing, ACID across docs, LINQ support
- ▶ Use when: team is C# / .NET; need transactional safety
- ▶ Firebase Firestore — best for mobile real-time
- ▶ Use when: mobile app; need automatic push updates



MODULE 2

Schema Flexibility — Versioning & Evolutionary Design

What is Schema Flexibility?

- Schema exists — there is a logical structure and expected fields
- Schema is NOT enforced by the database engine itself
- Fields can be absent from some documents in the same collection
- Responsibility for consistency moves to the application code
- Different documents in one collection can have different shapes

Why We Need Schema Flexibility

- Modern agile teams release new features multiple times per day
- New features often require new fields in existing documents
- Old documents remain valid alongside new document structures
- Business requirements change faster than data migrations can run
- The database must support continuous delivery, not block it

The Problem with Rigid Schemas

- Every structural change requires an ALTER TABLE command
- ALTER TABLE on a 100-million-row table can take hours
- Table is often locked during migration — app goes offline
- All existing rows must be updated simultaneously
- Migration failure can corrupt millions of rows at once

Document Versioning

- Each document declares its own schema version number
- New documents receive the latest version number on creation
- Old documents retain their original version and structure
- Application code handles each version with appropriate logic
- Background migration gradually upgrades old documents over time

Document Versioning — Example

- Version 1: { name: 'Maria Papadopoulou', schema_version: 1 }
- Version 2: adds separate first_name and last_name fields
- Version 2 doc: { first_name: 'Maria', last_name: 'Papadopoulou', schema_version: 2 }
- App reads version: if 1, split name on space to get first/last
- schema_version is just a normal integer field — DB is unaware

Advantages of Document Versioning

- Zero downtime — no database-level migration required
- Old data remains valid — never invalidated by new versions
- Backward compatibility maintained through all schema changes
- Rollback is trivial — revert the application code, not the data
- Development velocity is preserved — ship features without migrations

Schema Design Patterns Overview

- Embedding: put related data inside the document — owner relationship
- Referencing: store an ID and look up separately — shared entities
- Polymorphism: different document shapes in one collection via type field
- Bucket pattern: group time-series events into time-range buckets
- Schema versioning: stamp every document with a `schema_version` number

```
{
  "order_id": 10045,
  "customer_id": 872,
  "date": "2026-03-23",
  "items": [
    {
      "product_name": "Wireless Mouse",
      "price": 25,
      "quantity": 1
    },
    {
      "product_name": "Keyboard",
      "price": 60,
      "quantity": 1
    }
  ]
}
```

```
{
  "order_id": 10045,
  "customer_id": 872,
  "items": [ {
    "product_id": "P1001",
    "quantity": 1 } ]
}

{
  "product_id": "P1001",
  "name": "Wireless Mouse",
  "price": 25,
  "category": "Electronics"
}
```



MODULE 3

Indexing & Querying — Performance at Scale

What is an Index?

- A data structure mapping field values to document locations on disk
- Allows the database to find documents without scanning the collection
- Analogous to a book's index: look up keyword, get the page number
- Stored separately from documents — maintained automatically by the DB
- Improves read speed; slightly slows write speed (index must be updated)

```
{  
  "student_id": "ECE202400123",  
  "name": "Maria Papadopoulou",  
  "department": "Electrical and Computer Engineering",  
  "email": "maria.papadopoulou@uth.gr"  
}
```

- **db.students.createIndex({ student_id: 1 }) MONGODB INDEX**

Why Indexes are Critical

▶ Without Index

- ▶ Performs a full collection scan
- ▶ Reads every document one by one
- ▶ Query time: $O(N)$ — linear growth
- ▶ 1 million documents $\rightarrow$ 1M comparisons

▶ With Index

- ▶ Consults pre-sorted B-tree structure
- ▶ Jumps directly to matching documents
- ▶ Query time: $O(\log N)$ — logarithmic
- ▶ 1 million documents $\rightarrow$ ~20 comparisons

```
{  
  "registration_id": "REG938201",  
  "student_id": "ECE202400123",  
  "course_id": "ECE460",  
  "semester": "Spring 2026",  
  "grade": null  
}
```

Find all registrations where student_id = "ECE202400123"

When to Create an Index

- Index fields used in frequent filter conditions (find queries)
- Index fields used in sort operations on large collections
- Index fields used for uniqueness constraints (email, username)
- Do NOT index every field — writes become unacceptably slow
- Profile slow queries first — index based on evidence, not guesses

Types of Indexes

- Single field: index on one field — email, student_id
- Compound: index on multiple fields together — city + age
- Text: full-text search index for searching inside string content
- Geospatial: indexes GPS coordinates for location-based queries
- Unique: enforces that no two documents share the same field value

Creating an Index

- Command: `db.students.createIndex({ email: 1 })`
- Collection: `students` — specifies which collection to index
- Field: `email` — the field the index will be built on
- Value `1`: ascending sort order (use `-1` for descending)
- DB builds and maintains the index automatically from this point

What is a Query?

- A structured request to the database for specific documents
- Expressed as a JSON object matching the shape of desired data
- The database evaluates conditions and returns matching documents
- Query planner automatically selects the best available index
- Results are returned as an array of JSON documents

For example, a query searching for students in a particular department might look like this:

```
db.students.find({ department: "Electrical and Computer Engineering" })
```

This command shows whether the query uses an index or performs a full collection scan, along with additional details about the execution strategy.

```
db.students.find({ department: "ECE" }).explain()
```

Query Using an Index

- Command: `db.students.find({ email: 'maria@example.com' })`
- The query planner detects the email index exists
- Traverses the B-tree index to find 'maria@example.com'
- Retrieves only the matching document's disk location
- Fetches the document directly — typically under 1 millisecond

Simple Query & Projection

- Find all students in Volos: `db.students.find({ city: 'Volos' })`
- Returns all documents where city exactly matches 'Volos'
- Projection: `db.students.find({ city: 'Volos' }, { name: 1, email: 1 })`
- Projection returns only name and email — all other fields stripped
- Less data sent over the network → faster response for mobile apps

Filtering with Operators

- `$gt / $lt`: greater than / less than — `{ age: { $gt: 20 } }`
- `$gte / $lte`: greater or equal / less or equal
- `$ne`: not equal — `{ status: { $ne: 'inactive' } }`
- `$in`: matches any value in array — `{ city: { $in: ['Volos','Athens'] } }`
- `$and / $or`: combine multiple conditions logically

- Question
- How would you write a query to find students who are older than 20 and live in either Volos or Athens?
- Answer
- `db.students.find({`
- `age: { $gt: 20 },`
- `city: { $in: ["Volos", "Athens"] }`
- `})`

This query filters students whose age is greater than 20 and whose city is either Volos or Athens.

Sorting and Limiting Results

- Sort ascending: `.sort({ name: 1 })` — alphabetical A to Z
- Sort descending: `.sort({ age: -1 })` — oldest first
- Limit results: `.limit(10)` — return only the first 10 documents
- Skip for pagination: `.skip(20).limit(10)` — page 3 of 10 per page
- Combined: `db.students.find().sort({ name: 1 }).limit(20)`

What Queries Do NOT Do

- Queries do NOT join across multiple collections automatically
- Queries do NOT restructure document shape at retrieval time
- If frequent cross-collection joins are needed, schema is wrong
- Application-level joins: fetch from A, fetch from B, merge in code
- Design the document to match query needs — not the reverse

Common Querying Mistakes

- Indexing every field: cripples write performance with too many B-trees
- Fetching all documents and filtering in the application code
- Sorting in application memory instead of using the database
- Ignoring query explain plans — never verifying an index is used
- Schema designed without knowing the access patterns first



MODULE 4

Aggregation & Analytics — Computing Results from Data

What is Aggregation?

- Process multiple documents together and produce a computed result
- Not a data retrieval operation — a computation operation
- Produces new synthetic data: counts, averages, sums, groups
- Results are not stored in the database unless explicitly requested
- The foundation of dashboards, reports, and business analytics

The Aggregation Pipeline

- A pipeline is a sequence of processing stages
- Documents flow through each stage, transformed step by step
- Each stage receives the output of the previous stage
- Order of stages matters significantly for performance
- Syntax: `db.students.aggregate([ stage1, stage2, stage3 ])`

Core Pipeline Stages

- ▶ `$match` — filter documents (like find)
- ▶ `$group` — group by field, apply accumulators
- ▶ `$project` — reshape: include, exclude, compute fields
- ▶ `$sort` — sort the result set
- ▶ `$limit` / `$skip` — paginate aggregation results

- ▶ `$unwind` — flatten array fields into separate documents
- ▶ `$lookup` — left outer join with another collection
- ▶ `$count` — count matching documents into one result
- ▶ `$out` — write result to a new collection
- ▶ `$addFields` — add computed fields without removing existing ones

Aggregation Example: Count by City

- Goal: count how many students are in each city
- Stage 1 — \$group: group by city field, count documents
- { \$group: { _id: '\$city', total_students: { \$sum: 1 } } }
- Result: [{ _id: 'Volos', total_students: 120 }, { _id: 'Athens', total_students: 200 }]
- One pipeline stage replaces complex application-level logic

Aggregation Example: Average Grade

- Goal: find average grade per course across all students
- Stage 1 — \$unwind: flatten courses array into one doc per course
- Stage 2 — \$group: group by course name, compute \$avg of grade
- { \$group: { _id: '\$courses.name', avgGrade: { \$avg: '\$courses.grade' } } }
- Result: [{ _id: 'NoSQL Databases', avgGrade: 7.8 } ...]

When to Avoid Aggregations

- Aggregations are NOT a replacement for analytics or BI systems
- Do not run heavy aggregations on every user-facing page load
- Avoid aggregations that scan millions of documents in real-time
- Alternative: pre-compute and store results as summary documents
- For very large workloads, use dedicated data warehouses (BigQuery, Redshift)



MODULE 5

APIs — The Foundation of Modern Communication

What is an API?

- API: Application Programming Interface
- A set of rules allowing different software systems to communicate
- Frontends never query the database directly
- Frontends send requests to the backend API
- The backend translates those requests into database operations

Main Characteristics of REST APIs

- Stateless: every request contains all information needed — no sessions
- Resource-based: data treated as named resources (Student, Course)
- Uniform Interface: consistent rules applied to all resources
- JSON format: data exchanged as lightweight, human-readable text
- Cacheable: GET responses can be cached by browsers and CDNs

CRUD Operations Mapped to HTTP

▶ Create → POST /students

▶ Read → GET /students/123

▶ Update (full) → PUT /students/123

▶ Update (partial) → PATCH
/students/123

▶ Delete → DELETE /students/123

▶ POST sends new data in the request
body

▶ GET retrieves — never modifies server
data

▶ PUT replaces the entire document

▶ PATCH updates only the specified fields

▶ DELETE removes the resource
permanently

API Request: Path & Query Parameters

▶ Path Parameters

- ▶ Embedded directly in the URL path

- ▶ Identify one specific resource uniquely

- ▶ Example: `/api/students/12345`

- ▶ Used for: GET, PUT, PATCH, DELETE by ID

▶ Query Parameters

- ▶ Appended after `?` in the URL

- ▶ Filter, sort, search, or paginate a list

- ▶ Example:
`/api/students?city=Volos&sort=name`

- ▶ Used for: optional refinements on collections

API Request: Body & Headers

▶ Request Body (Payload)

- ▶ Main data sent with POST, PUT, or PATCH
- ▶ Contains complex nested JSON structures
- ▶ Transmitted securely — not visible in URL
- ▶ Can carry megabytes of structured data safely

▶ Request Headers

- ▶ Metadata about the request itself
- ▶ Authorization: Bearer <token> for security
- ▶ Content-Type: application/json signals format
- ▶ Accept: tells server what format client expects

HTTP Response Status Codes

- 200 OK — request succeeded, data returned in body
- 201 Created — new resource saved successfully (POST)
- 400 Bad Request — client sent invalid or malformed data
- 401 Unauthorized — missing or invalid authentication token
- 404 Not Found — requested resource does not exist in DB

API Authentication Methods

- API Key: a static secret passed in a header — simple but less secure
- JWT (JSON Web Token): signed token encoding user identity and roles
- OAuth 2.0: delegated access — Sign in with Google / GitHub
- Basic Auth: username:password in Base64 — only safe over HTTPS
- Bearer Token: JWT sent as Authorization: Bearer <token> header



MODULE 6

Schema Design, APIs & Microservices

Schema Design Philosophy

- Schema design does NOT start from tables or normalisation
- Schema design does NOT start from entity relationships
- Schema design starts from the API request the app will make
- Key question: what does this endpoint need to return?
- Design the document to answer that question in one query

Access Patterns

- Access pattern: how and how often the application reads data
- Which fields are requested together in the same API call?
- How often is this read performed — once per day or 1000 times per second?
- Is the data read more often than it is written?
- Design schema and indexes to serve the most frequent access patterns

Embedding vs Referencing

▶ Embedding

- ▶ Data stored inside the parent document
- ▶ Single query returns everything
- ▶ Best for: data owned by one parent
- ▶ Best for: data always read together

▶ Referencing

- ▶ Store ID — look up in separate collection
- ▶ Requires multiple queries or \$lookup
- ▶ Best for: shared data used by many parents
- ▶ Best for: data updated independently

Embedding — When and Why

- Use embedding when data belongs to exactly one parent document
- Use embedding when the data is always read together with the parent
- Use embedding when the data has the same lifecycle as the parent
- Example: student's course list is exclusively owned by the student
- Result: one query returns the student and all their courses

Referencing — When and Why

- Use references when data is shared across many parent documents
- Use references when child data is updated independently of the parent
- Use references when embedded arrays would grow without bound
- Example: course catalog exists independently of any student
- Result: two queries needed — but each entity is clean and independent

Web App Pattern: One Request, One Document

- Every web request expects a fast, complete response
- Ideal: one HTTP request → one database query → one complete document
- Design the document shape to match the frontend's exact data need
- Example: GET /api/students/12345 returns student + courses in one document
- Extra fields in the document are stripped via projection — no extra cost

What are Microservices?

- An architecture where the system is built as many small, independent services
- Each service has a single, well-defined responsibility
- Services communicate with each other via APIs — never via shared databases
- Each service is independently deployable and scalable
- Failure in one service does not cascade to bring down the entire system

Microservices & Document Databases

- Each microservice owns its own database — no sharing
- Student Service: owns students_db with student documents
- Enrollment Service: owns enrollments_db with enrollment documents
- The two services never query each other's database directly
- Services communicate exclusively via REST API calls

API Gateway Pattern

- API Gateway: a single entry point that composes responses from multiple services
- Frontend calls the gateway — not individual microservices directly
- Gateway calls Student Service and Enrollment Service in parallel
- Gateway merges the results in memory and returns one unified response
- The join happens in application code — never in the database



LAB SESSION

Build a REST API with FastAPI, MongoDB & Docker

Setting Up Your Python Environment

- Create a virtual environment: `python -m venv venv` (isolates packages per project)
- Activate — macOS/Linux: `source venv/bin/activate` | Windows: `venv\Scripts\activate`
- Install all dependencies: `pip install fastapi motor pydantic uvicorn`
- Verify: `uvicorn --version` (should print the installed version number)
- If pip is outdated: `python -m pip install --upgrade pip` then re-run install

LAB: What We Are Building

- Goal: a working CRUD REST API connected to a real MongoDB database
- The API creates, reads, and deletes student records stored as JSON documents
- Any client — Postman, a browser, a mobile app — can talk to our API over HTTP
- Everything runs locally inside Docker — no cloud account or installation needed
- End result: one running microservice you could deploy to the cloud today

Postman — Your API Testing Tool

- Postman is a desktop app for sending any HTTP request to any URL
- A browser's address bar can only make GET requests — it cannot send a JSON body
- In Postman: choose method → enter URL → add JSON body → click Send
- The response panel shows the returned JSON and the HTTP status code
- Alternative: FastAPI's built-in Swagger UI at <http://localhost:8000/docs>

Docker Compose — Running MongoDB

```
docker-compose.yml
1  version: '3.8'
2  services:
3    mongo:
4      image: mongo
5      ports:
6        - '27017:27017'
```

docker compose up -d → starts MongoDB in the background | docker compose down → stops it

FastAPI & Uvicorn — How They Work Together

- Uvicorn: the web server — opens port 8000 and listens for HTTP connections
- FastAPI: the framework — maps each URL and method to a Python function
- Motor: the database driver — sends queries to MongoDB and returns results
- Pydantic: the data validator — checks incoming JSON has the right fields and types
- Start everything: `uvicorn main:app --reload` (`--reload` restarts on file save)

Pydantic Models — Defining Data Shapes

```
models.py

1  from pydantic import BaseModel
2
3  class Student(BaseModel):
4      student_id: str
5      name: str
6      grade: int
7
8  # FastAPI validates incoming JSON automatically
9  # .model_dump() -> {'student_id': '101', 'name': 'Ana', 'grade': 3}
```

Wrong type or missing field → FastAPI returns 422 Unprocessable Entity before the DB is ever touched

Step 3: Create & Read Endpoints

```
main.py

1  @app.post('/students/')
2  async def create(s: Student):
3      await db.students.insert_one(s.model_dump())
4      return {'status': 'created'}
5
6  @app.get('/students/{student_id}')
7  async def read(student_id: str):
8      doc = await db.students.find_one({'student_id': student_id})
9      return doc
```

await pauses the function without blocking other requests — the server stays responsive while MongoDB replies

Step 4: Delete & Testing with Postman

- `DELETE /students/{student_id}` — `await db.students.delete_one({'student_id': id})`
- Start the server: `uvicorn main:app --reload` then open Postman
- CREATE: method POST → URL `localhost:8000/students/` → Body: raw JSON → Send
- READ: method GET → URL `localhost:8000/students/101` → Send
- DELETE: method DELETE → URL `localhost:8000/students/101` → Send



Thank You

Questions & Discussion

Advanced Data Management — Week 6

Bachelor of Computer Science